Μάθημα: Cyber Security
Εργασία: Security 2
Όνομα :Ρουμελιώτης Κωνσταντίνος
AM:ap25012
Επιβλέπων: Παναγιώτης Ριζομυλιώτης

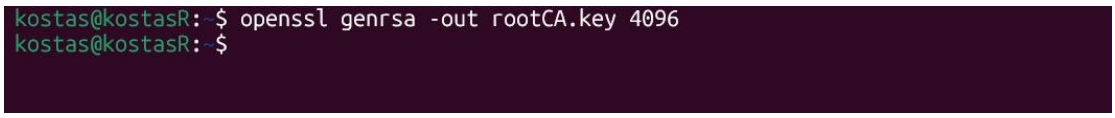
Ερώτηση 1



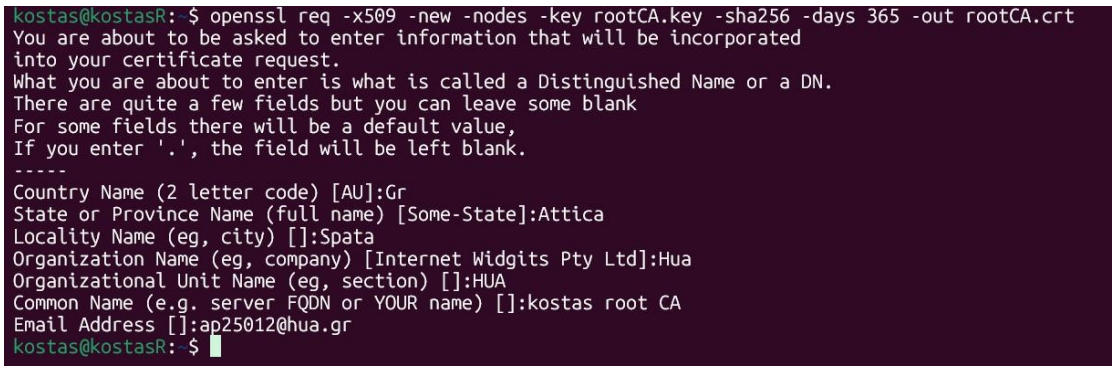
Στην αρχή τρέξαμε την εντολή για να ενημερωθεί το Ubuntu και να εγκατασταθούν τα εργαλεία openssl και curl, για να γίνει η εργασία.



Στην συνέχεια εκτελέσαμε την εντολή openssl version για να δούμε ότι η εγκατάσταση του προγράμματος έγινε. Παράλληλα, ελέγξαμε ποια ακριβώς έκδοση έχουμε στο σύστημά μας η οποία είναι η 3.0.13.



Στο συγκεκριμένο βήμα δημιουργήσαμε το ιδιωτικό κλειδί (private key) για την Αρχή Πιστοποίησής μας (Root CA). Εκτελώντας την εντολή genrsa, δημιουργήσαμε ένα κλειδί βασισμένο στον αλγόριθμο RSA και το αποθηκεύσαμε τοπικά στο αρχείο rootCA.key. Παράλληλα, ορίσαμε το μέγεθος του κλειδιού στα 4096 bits, εξασφαλίζοντας έτσι ένα πολύ υψηλό επίπεδο κρυπτογράφησης-ασφάλειας για το πιστοποιητικό.

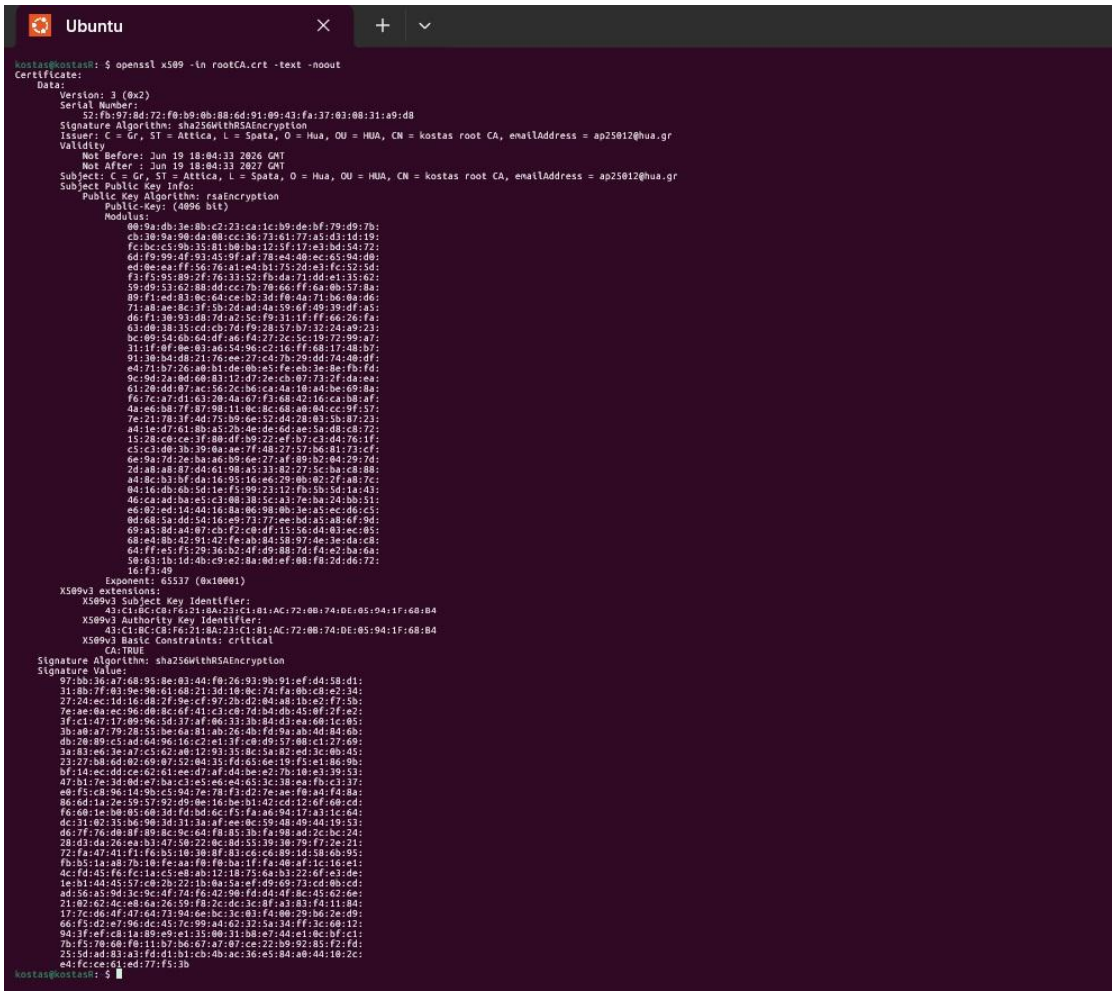


Σε αυτό το βήμα προχωρήσαμε στη δημιουργία του δικού μας πιστοποιητικού, το

οποίο επειδή το εκδώσαμε εμείς για τον εαυτό μας και λειτουργεί ως η επίσημη "ταυτότητα" της Αρχής Πιστοποίησής μας. Χρησιμοποιώντας την εντολή req -x509 και το ιδιωτικό κλειδί που είδη φτιάξαμε, δημιουργήσαμε απευθείας το τελικό πιστοποιητικό (rootCA.crt) με διάρκεια ισχύος 365 ημερών. Κατά την εκτέλεση της εντολής, το σύστημα μας ζήτησε να συμπληρώσουμε τα απαραίτητα στοιχεία ταυτότητας του πιστοποιητικού πράγμα που έγινε όπως π.χ τον οργανισμό (Hua) και κυρίως το όνομα της Αρχής μας (Common Name: kostas root CA).

```
kostas@kostasR:~$ ls -l rootCA*
-rw-r--r-- 1 kostas kostas 2106 Jun 19 21:04 rootCA.crt
-rw------- 1 kostas kostas 3268 Jun 19 21:00 rootCA.key
kostas@kostasR:~$
```

Εδώ εκτελέσαμε την εντολή ls -l για να δούμε όντως ότι δημιουργήθηκαν τα αρχεία μας επιτυχώς. Όπως φαίνεται στο αποτέλεσμα, τόσο το ιδιωτικό κλειδί (rootCA.key) όσο και το πιστοποιητικό της Αρχής μας (rootCA.crt) έχουν αποθηκευτεί κανονικά στον φάκελο μας.



Εδώ χρησιμοποιήσαμε την εντολή openssl x509 -text για να μπορέσουμε να "διαβάσουμε" το περιεχόμενο του πιστοποιητικού που μόλις δημιουργήσαμε. Επειδή

το αρχείο κανονικά είναι σε μη αναγνώσιμη μορφή, η εντολή αυτή το μεταφράζει σε απλό κείμενο. Έτσι, μπορέσαμε να ελέγξουμε στην οθόνη μας και να επιβεβαιώσουμε όλα τα στοιχεία που είχαμε δηλώσει πχ την ημερομηνία λήξης του.

```
kostas@kostasR: $ openssl ecparam -name prime256v1 -genkey -noout -out ecdsa.key
kostas@kostasR: $
```

Σε αυτό το σημείο προχωρήσαμε στη δημιουργία ενός νέου ιδιωτικού κλειδιού. Η διαφορά εδώ είναι ότι, αντί για τον κλασικό αλγόριθμο RSA που χρησιμοποιήσαμε στην πρώτη φορά το φτιάξαμε με βάσει το ECDSA. Αυτή η μέθοδος είναι πιο "ελαφριά" και γρήγορη, προσφέροντας υψηλή ασφάλεια. Το νέο κλειδί αποθηκεύτηκε στο αρχείο ecdsa.key .

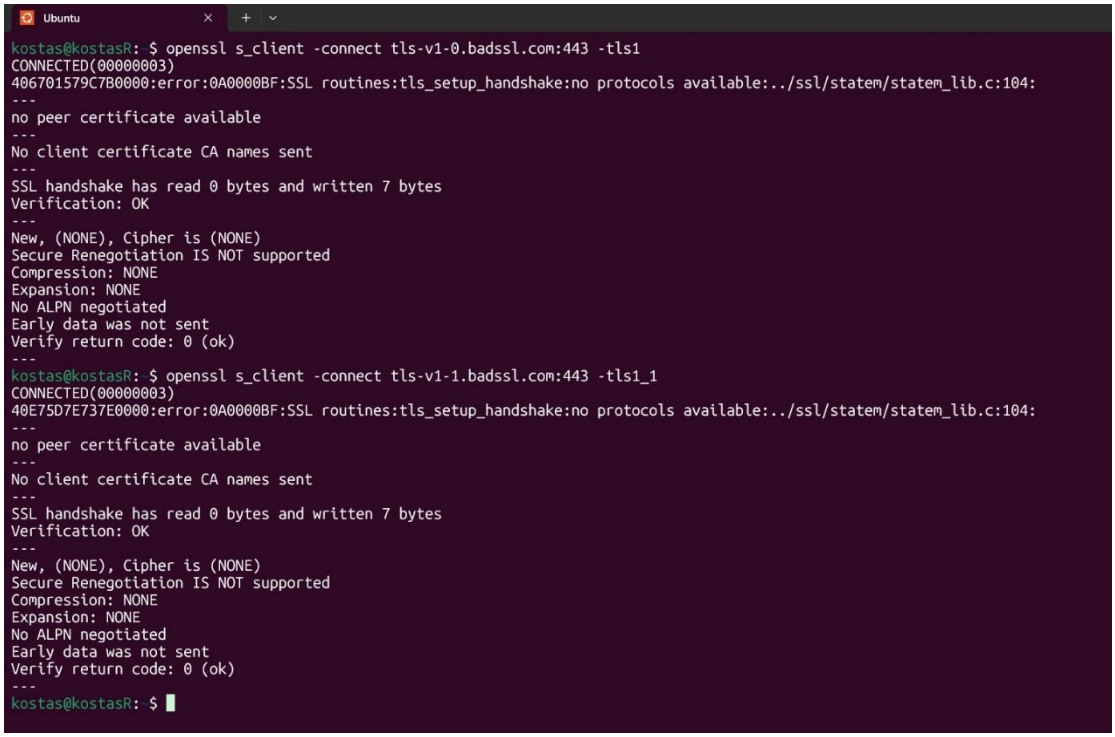
```
kostas@kostasR: $ openssl req -new -key ecdsa.key -out ecdsa.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:Gr
State or Province Name (full name) [Some-State]:Attica
Locality Name (eg, city) []:Spata
Organization Name (eg, company) [Internet Widgits Pty Ltd]:hua
Organizational Unit Name (eg, section) []:HUA
Common Name (e.g. server FQDN or YOUR name) []:kostas ECDSA Cert
Email Address []:ap25012@hua.gr

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:1234
An optional company name []:kr
kostas@kostasR: $
```

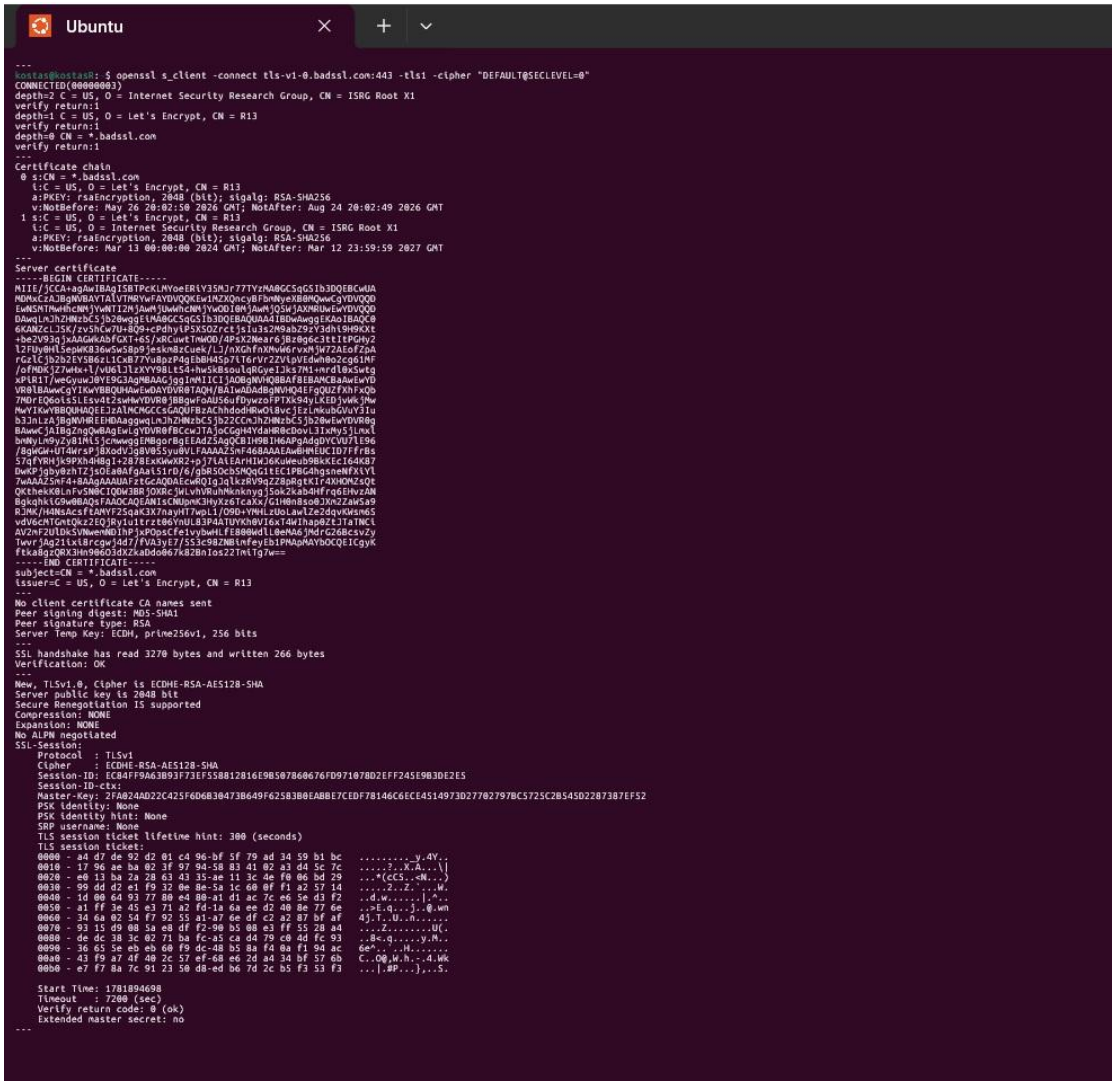
```
kostas@kostasR: $ openssl x509 -req -in ecdsa.csr -CA rootCA.crt -CAkey rootCA.key -CAcreateserial -out ecdsa.crt -days 365 -sha256
Certificate request self-signature ok
subject=C = Gr, ST = Attica, L = Spata, O = hua, OU = HUA, CN = kostas ECDSA Cert, emailAddress = ap25012@hua.gr
kostas@kostasR: $
```

Σε αυτά τα δύο βήματα ολοκληρώνεται η διαδικασία έκδοσης του νέου μας πιστοποιητικού. Αρχικά, χρησιμοποιώντας το κλειδί ECDSA, δημιουργήσαμε μια "Αίτηση Υπογραφής" (CSR - Certificate Signing Request), συμπληρώνοντας τα απαραίτητα στοιχεία ταυτοποίησης (όνομα, περιοχή, email) όπως και την πρώτη φόρα. Στη συνέχεια το πήραμε αυτό και το δώσαμε στη δική μας Αρχή Πιστοποίησης (Root CA) που φτιάξαμε στην αρχή. Η Αρχή μας έλεγξε την αίτηση, την υπέγραψε ψηφιακά και εκδόθηκε το τελικό, έγκυρο πιστοποιητικό μας (ecdsa.crt) που έχει ισχύ για ένα χρόνο.

Ερώτημα 2



Σε αυτό το ερώτημα, προσπαθήσαμε να συνδεθούμε στη δοκιμαστική ιστοσελίδα badssl.com ζητώντας να χρησιμοποιήσει τα πρωτόκολλα TLS 1.0 (πρώτη εντολή) και TLS 1.1 (δεύτερη εντολή). Όπως βλέπουμε στα αποτελέσματα, και οι δύο προσπάθειες απέτυχαν, εμφανίζοντας το σφάλμα "no protocols available". Αυτό συμβαίνει επειδή το σύστημά μας χρησιμοποιεί τη σειρά 3.x του OpenSSL. Όπως είχαμε διαπιστώσει κατά τον αρχικό έλεγχο της έκδοσης (openssl version), αυτή η εκδοχή έχει ρυθμιστεί να μπλοκάρει αυτόματα από προεπιλογή τα ξεπερασμένα πρωτόκολλα για λόγους ασφαλείας για αυτό και τα σφάλματα σύνδεσης που συναντήσαμε εδώ κατά την εκτέλεση των δύο εντολών.


Ubuntu
kostas@kostasR: $ openssl s_client -connect tls-v1-0.badssl.com:443 -tls1 -cipher "DEFAULT@SECLEVEL=0"
CONNECTED(00000003)
depth=2 C = US, O = Internet Security Research Group, CN = ISRG Root X1
verify return:1
depth=1 C = US, O = Let's Encrypt, CN = R13
verify return:1
depth=0 CN = *.badssl.com
verify return:1

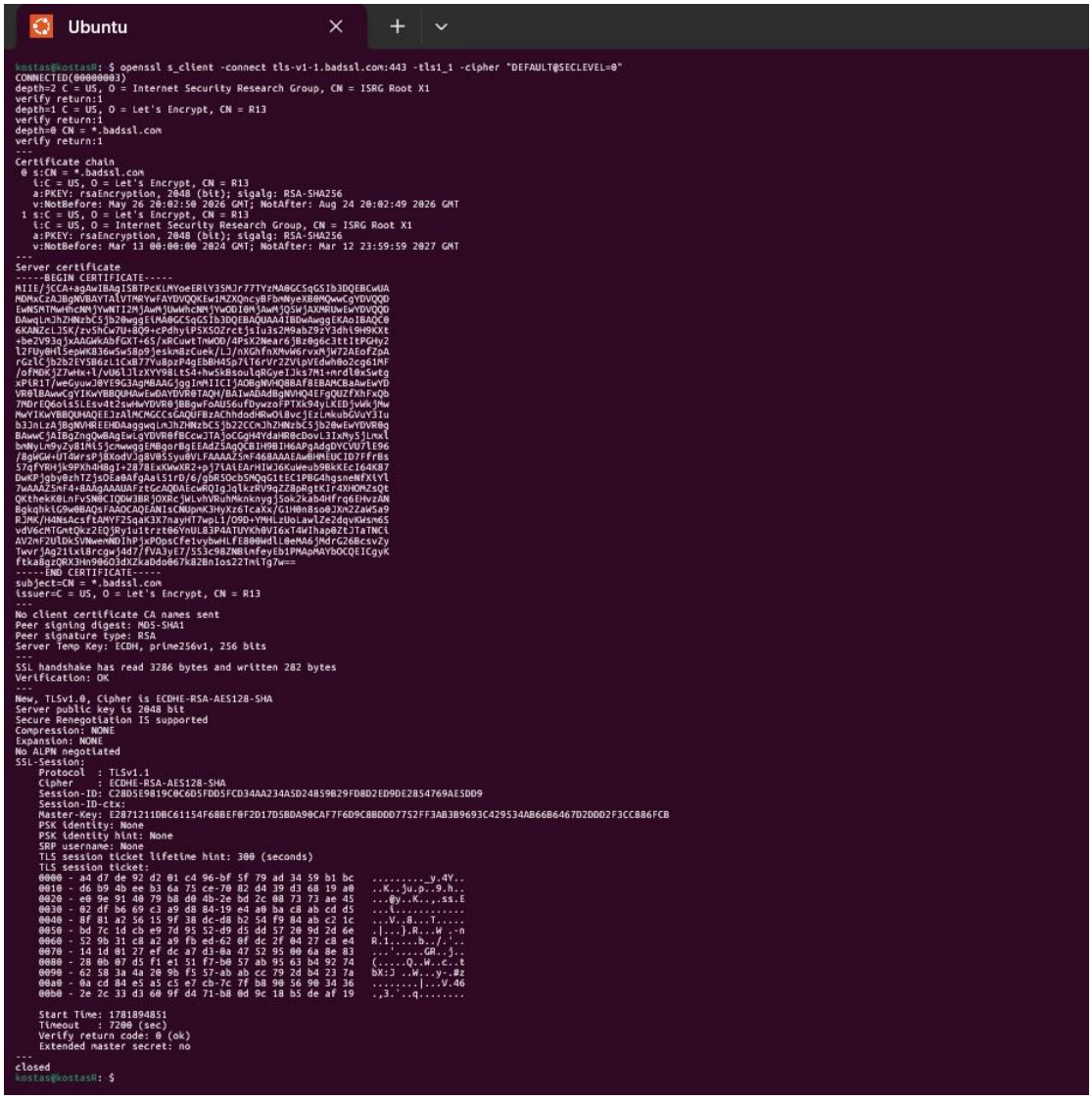
Certificate chain
0 s:CN = *.badssl.com
i:C = US, O = Let's Encrypt, CN = R13
a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
v:NotBefore: May 26 20:02:50 2026 GMT; NotAfter: Aug 24 20:02:49 2026 GMT
1 s:C = US, O = Let's Encrypt, CN = R13
i:C = US, O = Internet Security Research Group, CN = ISRG Root X1
a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
v:NotBefore: Mar 13 00:00:00 2024 GMT; NotAfter: Mar 12 23:59:59 2027 GMT

Server certificate
-----BEGIN CERTIFICATE-----
MIIE/jCCA+agAwIBAgISBTPcKLMYoeERlV3SMJr77YzMA0GCSqGSIb3DQEBCwUA
MDMxCzAJBgNVBAYTAlVTMRYwFAYDVQQKEw1MZXQncyBFbmNyeXB0MQwwCgYDVQQD
EwNSMTMwHhcNMjYwNTI2MjAwMjUwWhcNMjYwODI0MjAwMjQ5WjAXMRUwEwYDVQQD
DAwqLmJhZHNzbC5jb20wggEiMA0GCSqGSIb3DQEBAQUAA4IBDwAwggEKAoIBAQC0
6KANZcLJSK/zvShCw7U+8Q9+cPdhylPSXSOZrctjslu3s2M9abZ9zY3dhi9H9KXt
+be2V93qjsAAGWkAbfGXT+6S/xRCuwtYmKOD/dP1X2Wear6JBz8g6c3ttIfPGHy2
l2fUy0HlSepWKB36wSwS8p9jeskm8zCuek/LJ/nXGhfnXMvW6rvxMjW72AEofZpA
rGzlCjb2b2EYSB6zL1CxB77YvBpzP4gEbBH4Sp7lT6rVr2ZVlpVEdwh0o2cg61MF
/oFMD6jZ7wHx+l/yW6l3LbXYY9BLt54+hw5kB8soUlqRGyeI3ks7Mi+vrdl0xSwtg
xPiR1I/weGyuwJ0YE9GJAgMBAAGjggImMIICJAOBgNVHQ8BAf8EBAMCBaAwEwYD
VR0lBAwwCgYIKwYBBQUHAwEwDAYDVR0TAQH/BAIwADAdBgNVHQ4EFgQUZfXhFxQb
7MDrEQ6oissLEsv4t2swHwYDVR0jBBgwFoAU56uFDywzoFPTXk94yLKED3vWkjMw
MwYIKwYBBQUHAQEEJzAlMCMGCCsGAQUFBzAChhdodHRwOi8vZi5pLmkubGVuY3Iu
b3JnLzAjBgNVHREEHDAaggwqLmJhZHNzbC5jb22CCnJhZHNzbC5jb20wEwYDVR0g
BAwwCjAIBgZngQwBAgEwLgYDVR0fBCcwJTAjoCGgH4YdaHR0cDovL3IxMy5jLmxl
bnNyLm9yZy8IRlS5jcmwwggEMBgorBgEEAdZ5AgQCBIH9BIH6APgAdgDVCVU7lE96
/8gWGW+UT4WrsPJ8XodVJgBVB5yu8VLFAAAAZ5nF46BAAAEAwBHMEUCID7FfrBs
57qfYRHjk9PXh4H8g1+2878ExKWwXR2+pj7lAlEArHIWJ6KuWeub9BkKEc164KB7
BwKPJgby0zhTZjsOEa0AfgAaAS1rD/6/gbRSOcbSMQgGitEC1PBGdhqsneNfXlYl
7wAAZ5nF4+BAAqAAUAJFtGcAQDAEcwRQIgJolkzRV9gZZ8pRgtKIr4XHXMZsQt
QKthekK0lnFvSN0CIQDW3BRjOXRcjWLvhVRuhMknkvgjSok2kab4HFrq6EhvzAN
BgkqhkiG9w0BAQsFAAOCAQEANIsCNUpnK3HyXz6TcaXx/G1H0n8soQJXn2ZaWSa9
RJMK/H4NsAcsftAMYF2SqaK3X7nayHI7wpLi/O9D+YMHLzUoLaWlZe2dqvKWsmG5
vdv6cMIGmtQKz2tQjRyiu1trzt0GYoUL83PAIUYKh0V16xI4WIha98ZtJTaTNCL
AV2mF2UlDkSVNwemNDIhPjxPOpsCfe1vybwHLfE800WdlL0eMA6jMdrG26Bcsv2y
TwvrjAg21ixl8rcgwj4d7/fVA3yE7/SS3c98ZNBimfeyEb1PMAqMAYbOCQEICgyK
fkaBgzQRX3Hn9O6OJdXZkaDdo067k82Bn1os22TmIg7w==
-----END CERTIFICATE-----
subject=CN = *.badssl.com
issuer=C = US, O = Let's Encrypt, CN = R13

No client certificate CA names sent
Peer signing digest: MD5-SHA1
Peer signature type: RSA
Server Temp Key: ECDH, prime256v1, 256 bits

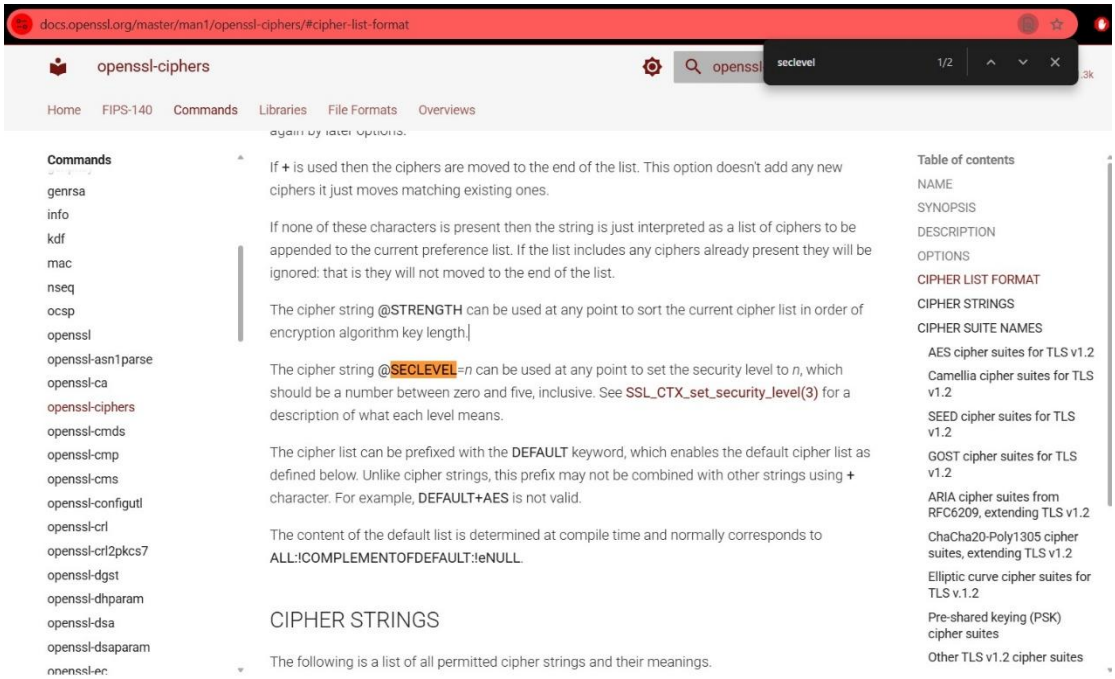
SSL handshake has read 3270 bytes and written 266 bytes
Verification: OK

New, TLSv1.0, Cipher is ECDHE-RSA-AES128-SHA
Server public key is 2048 bit
Secure Renegotiation IS supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
SSL-Session:
Protocol : TLSv1
Cipher : ECDHE-RSA-AES128-SHA
Session-ID: ECB4FF9A63B93F73EF5588121816E9B507860676FD97107BD2EFF245E9B3DE2E5
Session-ID-ctx:
Master-Key: 2FA024AD22C425F6D6B30473B649F625B3B0EABBE7CEDF78146C6ECE4514973D27702797BC5725C2B545D22873B7EF52
PSK identity: None
PSK identity hint: None
SRP username: None
TLS session ticket lifetime hint: 300 (seconds)
TLS session ticket:
0000 - a4 d7 de 92 d2 01 c4 96-bf 5f 79 ad 34 59 b1 bc ...._y.4Y..
0010 - 17 96 ae ba 02 3f 97 94-58 83 41 02 a3 d4 5c 7c .....?..X.A...\|
0020 - e0 13 ba 2a 28 63 43 35-ae 11 3c 4e f0 06 bd 29 ...*(cC5..<N...)
0030 - 99 dd d2 e1 f9 32 0e 8e-5a 1c 60 0f f1 a2 57 1d .....2..Z.`...W.
0040 - 1d 00 64 93 77 80 e4 00-a1 d1 ac 7c e6 5e d3 f2 ..d.w......|.^..
0050 - a1 ff 3e 45 e3 71 a2 fd-1a 6a ee d2 40 8e 77 6e ..>E.q...j..@.wn
0060 - 34 6a 02 54 f7 92 55 a1-a7 6e df c2 a2 87 bf af 4j.T..U..n......
0070 - 93 15 d9 08 5a e8 df f2-90 b5 08 e3 ff 55 28 a4 ....Z........U(.
0080 - de dc 38 3c 02 71 ba fc-a5 ca dd 79 c0 4d fc 93 ..8<.q.....y.M..
0090 - 36 65 5e eb eb 60 f9 dc-48 b5 8a f4 0a f1 94 ac 6e^..`..H.......
00a0 - 43 f9 a7 4f 40 2c 57 ef-68 e6 2d a4 34 bf 57 6b C..O@,W.h.-.4.Wk
00b0 - e7 f7 8a 7c 91 23 50 d8-ed b6 7d 2c b5 f3 53 f3 ...|.#P...},..S.
Start Time: 1781894698
Timeout : 7200 (sec)
Verify return code: 0 (ok)
Extended master secret: no



Σε αυτές τις δύο εικόνες ξανά εκτελούμε τις δύο παραπάνω εντολές για την προσπάθεια σύνδεσης με τα ξεπερασμένα πρωτόκολλα (TLS 1.0 και TLS 1.1), αλλά στο τέλος της κάθε εντολής βάλαμε την ειδική παράμετρο -cipher "DEFAULT@SECLEVEL=0". Με αυτόν τον τρόπο λέμε στο OpenSSL να ρίξει το επίπεδο ασφαλείας του στο μηδέν (Security Level 0), απενεργοποιώντας έτσι τον προεπιλεγμένο αποκλεισμό που είδαμε πριν. Όπως παρατηρούμε στα αποτελέσματα, η παράκαμψη λειτούργησε και το αρχικό μήνυμα CONNECTED επιβεβαιώνει ότι η σύνδεση πραγματοποιήθηκε πλέον με επιτυχία και στις δύο περιπτώσεις, επιτρέποντας στο σύστημά μας να διαβάσει κανονικά τα πιστοποιητικά της δοκιμαστικής σελίδας.

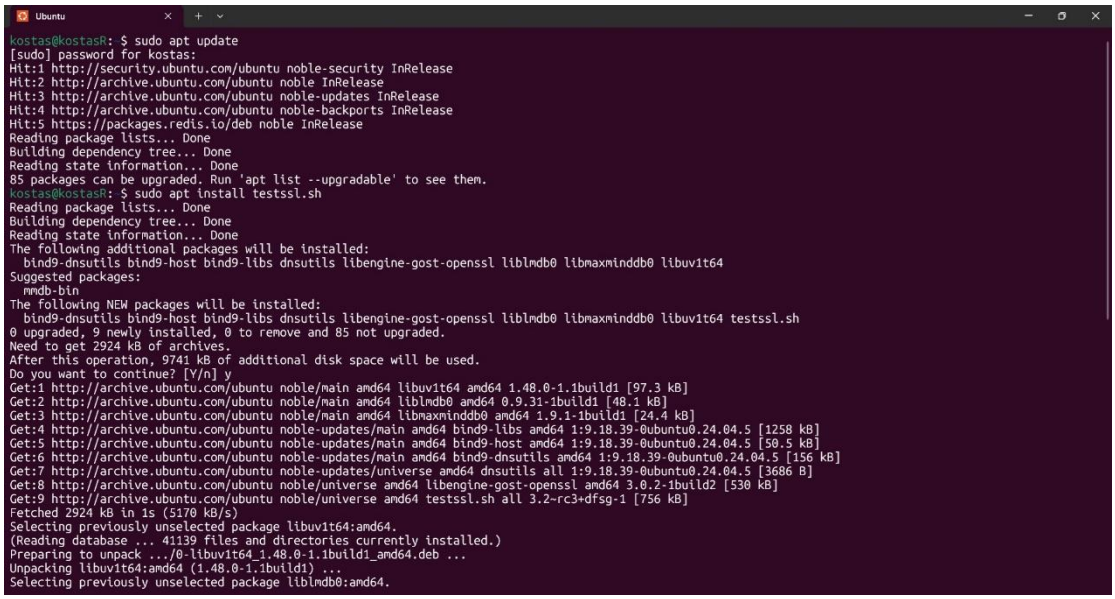
** Η τροποποίηση της εντολής για να γίνει η παράκαμψη βρέθηκε από το documentation του openssl το οποίο παρατίθεται στην παρακάτω εικόνα και εξηγεί την λογική.




```
kostas@kostasR: $ openssl s_client -connect sha1-2017.badssl.com:443 -showcerts
CONNECTED(00000003)
depth=0 C = US, ST = California, L = Walnut Creek, O = Lucas Garron, CN = *.badssl.com
verify error:num=68:CA signature digest algorithm too weak
verify return:1
depth=1 C = US, O = DigiCert Inc, CN = DigiCert Secure Server CA
verify error:num=68:CA signature digest algorithm too weak
verify return:1
depth=2 C = US, O = DigiCert Inc, OU = www.digicert.com, CN = DigiCert Global Root CA
verify return:1
depth=1 C = US, O = DigiCert Inc, CN = DigiCert Secure Server CA
verify error:num=10:certificate has expired
notAfter=Mar  8 12:00:00 2023 GMT
verify return:1
depth=1 C = US, O = DigiCert Inc, CN = DigiCert Secure Server CA
notAfter=Mar  8 12:00:00 2023 GMT
verify return:1
depth=0 C = US, ST = California, L = Walnut Creek, O = Lucas Garron, CN = *.badssl.com
verify error:num=10:certificate has expired
notAfter=Jan  5 12:00:00 2017 GMT
verify return:1
depth=0 C = US, ST = California, L = Walnut Creek, O = Lucas Garron, CN = *.badssl.com
notAfter=Jan  5 12:00:00 2017 GMT
verify return:1
---
Certificate chain
 0 s:C = US, ST = California, L = Walnut Creek, O = Lucas Garron, CN = *.badssl.com
   i:C = US, O = DigiCert Inc, CN = DigiCert Secure Server CA
   a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA1
   v:NotBefore: Jun  9 00:00:00 2015 GMT; NotAfter: Jan  5 12:00:00 2017 GMT
-----BEGIN CERTIFICATE-----
```

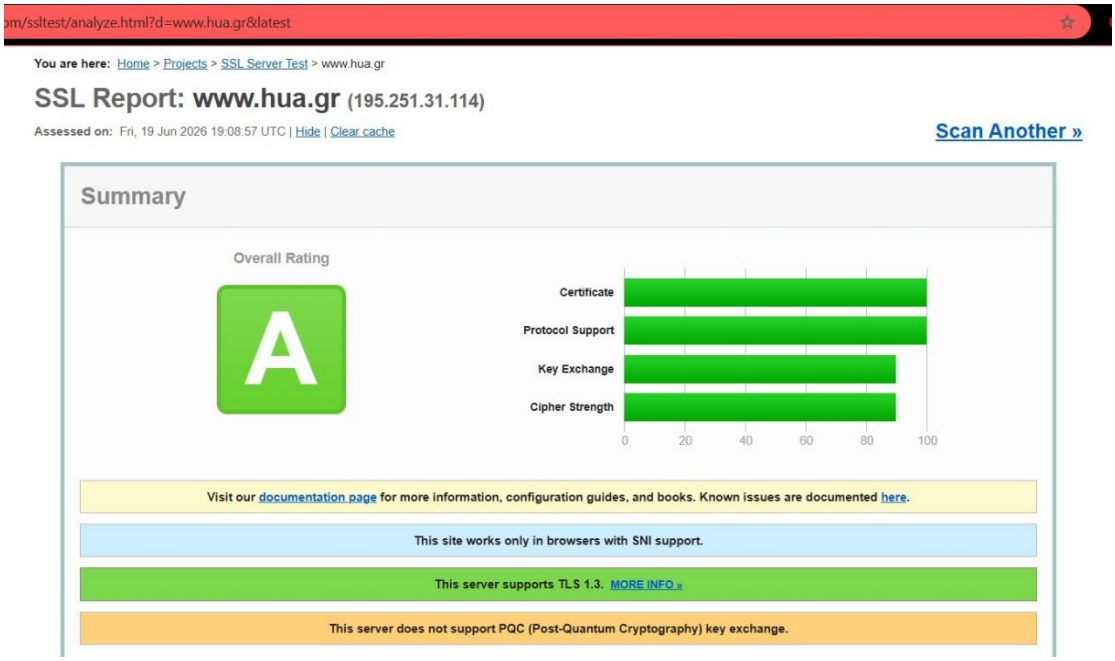
Σε αυτό το βήμα, συνδεθήκαμε σε μια άλλη δοκιμαστική σελίδα (τη sha1-2017.badssl.com) προσθέτοντας την παράμετρο -showcerts, η οποία ζητάει να μας εμφανιστεί όλη η αλυσίδα των πιστοποιητικών του server. Αρχικά σύνδεση έγινε (CONNECTED) στην συνέχεια όμως το σύστημά μας εντόπισε κινδύνους κατά τον έλεγχο της σελίδας. Όπως φαίνεται ξεκάθαρα στα αποτελέσματα, εντοπίστηκαν δύο πολύ σοβαρές αστοχίες. Για αρχή ο αλγόριθμος που έχει χρησιμοποιηθεί για την υπογραφή του πιστοποιητικού (SHA1) θεωρείται πλέον πολύ αδύναμος και εντελώς χωρίς μεγάλη ασφάλεια (CA signature digest algorithm too weak). Επίσης το σύστημα διαπίστωσε ότι τα πιστοποιητικά έχουν λήξει εδώ και πολλά χρόνια (certificate has expired). Σαν συμπέρασμα να πούμε ότι το OpenSSL ελέγχει και μας προειδοποιεί για επικίνδυνες ιστοσελίδες.

Ερώτημα 3



Στην αρχή εκτελέσαμε την εντολή sudo apt update για να ενημερώσουμε τη λίστα με τα διαθέσιμα προγράμματα του Ubuntu. Στη συνέχεια, δώσαμε την εντολή sudo apt install testssl.sh για να εγκαταστήσουμε το εργαλείο, το οποίο θα χρησιμοποιήσουμε για να αξιολογήσουμε την ασφάλεια ιστοσελίδων.





Σε αυτές τις δύο εικόνες βλέπουμε τα τελικά αποτελέσματα της αξιολόγησης ασφαλείας για την ιστοσελίδα www.hua.gr. Στην πρώτη εικόνα τρέξαμε το τοπικό εργαλείο testssl.sh μέσα από το τερματικό μας, ενώ στη δεύτερη χρησιμοποιήσαμε τη σελίδα της Qualys (SSL Labs). Όπως παρατηρούμε, τα αποτελέσματα ταυτίζονται απόλυτα αφού και στις δύο περιπτώσεις έχουμε τον βαθμό (A). Αν ήταν διαφορετικά αυτό θα οφειλόταν στο ότι η σελίδα-πλατφόρμα SSL Labs είναι πάντα ενημερωμένη "κεντρικά" με τις πιο πρόσφατες απειλές. Αντίθετα, το εργαλείο testssl.sh που κατεβάσαμε στον υπολογιστή μας μπορεί μετά από λίγους μήνες να παλιώσει. Πολλές μεγάλες ιστοσελίδες κρύβουν πολλούς servers πίσω από το ίδιο όνομα για να μην "πέφτουν". Το SSL Labs ελέγχει συνήθως *όλους* τους διαθέσιμους servers και σου δείχνει τον βαθμό του χειρότερου. Το testssl.sh ελέγχει συνήθως μόνο τον πρώτο server που θα του απαντήσει. Το SSL Labs "βλέπει" την ιστοσελίδα από την Αμερική, ενώ το δικό σου τερματικό τη βλέπει από το δίκτυο του σπιτιού. Αν η ιστοσελίδα έχει κάποιο Firewall που κόβει συνδέσεις ανάλογα με τη χώρα τα εργαλεία θα έχουν εντελώς διαφορετική εικόνα